

Capa de datos para un copiloto RAG interno de una distribuidora mayorista

Martín Madrid César Andrés Orellana Nicolás Valentín Ciarrapico

Trabajo Práctico Integrador – Bases de Datos para Inteligencia Artificial
Carrera de Especialización en Inteligencia Artificial – FIUBA
2026

Resumen

En este trabajo diseñamos la capa de datos de un copiloto RAG interno para una distribuidora mayorista. El problema que buscamos resolver es que la información relevante está repartida entre documentos, procedimientos, condiciones comerciales y datos operativos, por lo que encontrar una respuesta confiable puede ser lento y depender del conocimiento informal de las personas.

La solución propuesta usa PostgreSQL como motor principal, junto con las extensiones `pgvector` y `btree_gist`. En la misma base se guardan los datos operativos, los metadatos de los documentos, los fragmentos, los embeddings, los permisos y los eventos de auditoría. Elegimos este enfoque porque permite aplicar la vigencia y los permisos antes de buscar por similitud, lo que excluye del ranking las fuentes vencidas o no autorizadas.

La implementación incluye datos sintéticos reproducibles, siete consultas funcionales y dos pruebas transversales de seguridad y auditoría. Las restricciones de integridad, las políticas RLS y la evidencia asociada a cada respuesta permiten comprobar los contratos principales mediante dos cargas limpias equivalentes. El trabajo no implementa una interfaz, una API ni un modelo de lenguaje real; la prueba termina en la capa de datos.

Índice

1. Descripción del caso de uso	3
2. Relevamiento de datos necesarios	3
3. Clasificación de los datos según su tipo	4
4. Modelo conceptual	4
5. Modelo de implementación según la tecnología elegida	5
6. Decisiones de normalización, embebido, referencia y desnormalización	6
7. Justificación de la tecnología seleccionada	6
8. Implementación mínima realizada	7
9. Datos de ejemplo utilizados	7
10.Consultas representativas	8
11.Propuesta para datos semiestructurados, no estructurados y vectoriales	8
12.Propuesta de arquitectura de datos	9
13.Estrategia de seguridad, permisos y aislamiento	10
14.Consideraciones de escalabilidad y rendimiento	11
15.Conclusiones	12
A. Reproducibilidad y material de apoyo	13

1. Descripción del caso de uso

El caso elegido es el de un sistema RAG para consultar documentación técnica. Lo adaptamos al contexto de una distribuidora mayorista, ya que en este tipo de empresa la información está distribuida entre catálogos de productos, condiciones comerciales, pedidos, entregas, incidencias, procedimientos de logística y documentos de proveedores o de cumplimiento. En la práctica, una persona que necesita resolver una consulta debe buscar en varios documentos o preguntarle a alguien que conozca el proceso. Esto puede generar demoras, respuestas diferentes para una misma pregunta y uso de documentación que ya no está vigente.

Nuestra propuesta es diseñar la capa de datos que permitiría a un copiloto interno recuperar información relevante y usarla como fuente para responder. El alcance comprende el almacenamiento, la integridad, la autorización, la recuperación y la trazabilidad. El entrenamiento de modelos y la construcción de una aplicación completa quedan fuera de la prueba.

Los usuarios principales que consideramos son **Operaciones y Logística**, que consulta fichas de producto, procedimientos logísticos e incidencias autorizadas; **Comercial y Compras**, que consulta condiciones comerciales, políticas de venta e información autorizada de proveedores; y **Administración y Calidad**, que consulta documentación de cumplimiento, documentación legal y la trazabilidad de las respuestas. Además, consideramos servicios internos de ingesta, consulta y administración documental. Estos servicios no representan usuarios finales y tienen permisos mínimos según la tarea que realizan.

El sistema debe poder registrar documentos con versiones, determinar qué versión está vigente, dividir los documentos en fragmentos, asociar embeddings a esos fragmentos y recuperar los más cercanos a una consulta. También debe resolver consultas estructuradas, por ejemplo sobre pedidos, entregas o condiciones comerciales. Para cada respuesta exitosa se guarda la evidencia que indica qué fuente fue utilizada.

Los riesgos más importantes son recuperar contenido no autorizado, usar una versión sustituida o revocada, responder sin poder justificar la fuente y perder la trazabilidad de una consulta anterior. Por este motivo, las tres decisiones centrales del diseño son conservar el historial de versiones, aplicar permisos antes de la recuperación y guardar evidencia y auditoría de las interacciones.

El alcance del trabajo se limita a la capa de datos. No incluimos frontend, backend, API, autenticación de usuarios, integración real con un LLM ni embeddings generados por un modelo comercial. Estos componentes se consideran posibles consumidores futuros de la base diseñada.

2. Relevamiento de datos necesarios

Para diseñar la solución separamos los datos según el papel que tienen dentro del problema. No todos los datos tienen el mismo ciclo de vida ni se consultan de la misma forma. Por ejemplo, un pedido se consulta con filtros y relaciones exactas, mientras que un procedimiento escrito se recupera por similitud semántica.

Los datos operativos son necesarios porque el copiloto no debería resolver por similitud preguntas que tienen una respuesta exacta en la base. Por ejemplo, la condición comercial vigente se obtiene cruzando cliente, producto y fecha; los pedidos con entregas demoradas se obtienen relacionando pedidos, entregas e incidencias. Guardar estos datos de forma estructurada permite aplicar restricciones, realizar agregaciones y evitar ambigüedades.

Para el corpus documental necesitamos registrar más información que el texto. Cada documento tiene una clase, un nivel de sensibilidad, una procedencia y una o varias versiones. Cada versión conserva el archivo original mediante su ruta relativa, nombre, tipo MIME, tamaño y checksum SHA-256. La base verifica así la identidad del archivo sin guardar su contenido completo.

Cada versión se divide en fragmentos. El fragmento es la unidad que se vectoriza y también

Grupo	Datos principales	Uso dentro de la solución
Operativos	Productos, categorías, clientes, segmentos, proveedores, pedidos, líneas de pedido, entregas e incidencias.	Responder consultas exactas del negocio y relacionar documentos con el contexto operativo.
Comerciales	Condiciones comerciales, precios, descuentos y períodos de vigencia.	Consultar qué condición corresponde a un cliente y producto en una fecha determinada.
Documentales	Documentos, versiones, clases documentales, sensibilidad, archivos y fragmentos.	Mantener el corpus, su procedencia, su vigencia y la unidad de recuperación.
Vectoriales	Modelos de embedding y vectores de cada fragmento.	Resolver búsquedas por similitud sobre documentación autorizada.
Acceso	Actores, perfiles autorizados y permisos por clase documental.	Limitar qué documentos puede recuperar cada perfil.
Interacción y auditoría	Consultas, respuestas, evidencias y eventos de auditoría.	Explicar respuestas anteriores y registrar acciones relevantes.

Cuadro 1: Datos relevados para la solución.

la unidad que luego se puede mostrar como evidencia. Esto permite indicar con precisión qué sección de qué versión fue utilizada para fundamentar una respuesta, en lugar de asociar la respuesta a un documento completo.

Cada perfil puede ver un conjunto distinto de clases documentales. El permiso se modela como una relación entre perfil y clase, y la ausencia de esa relación significa denegación. Los datos de interacción y auditoría registran quién consultó, con qué perfil, qué resultado obtuvo y qué evidencia se utilizó, sin duplicar el contenido sensible de los documentos.

Para la implementación usamos datos sintéticos. Esto evita trabajar con datos personales, comerciales o documentos internos reales y permite repetir la carga con los mismos conteos, archivos, vectores y resultados esperados.

3. Clasificación de los datos según su tipo

La clasificación combina forma, función y sensibilidad. Una misma entidad puede pertenecer a más de una categoría: una consulta es un dato operacional durante la interacción y también alimenta análisis posteriores de uso.

Los datos analíticos no se persisten como una segunda fuente de verdad porque el volumen y las consultas del TP no lo requieren. Si el historial de uso o la auditoría alcanzaran una escala que afectara al núcleo transaccional, podrían replicarse en una capa analítica sin cambiar el modelo operativo.

4. Modelo conceptual

La Figura 1 presenta las entidades, relaciones y cardinalidades del dominio.

El modelo separa cuatro núcleos relacionados. El núcleo operativo reúne productos, clientes, condiciones comerciales, pedidos y entregas. El núcleo documental organiza documentos, versiones y fragmentos. El acceso vincula cada actor con un único perfil y cada perfil con las clases documentales permitidas. La interacción conserva consultas, respuestas, evidencias y eventos de auditoría.

Las cardinalidades expresan reglas del dominio. Un pedido pertenece a un cliente y contiene una o más líneas; una entrega pertenece a un pedido, que puede tener entregas parciales; una incidencia pertenece a una entrega. Un documento puede tener varias versiones y cada versión

Tipo	Ejemplos en la solución
Estructurados	Catálogos, productos, clientes, proveedores, condiciones comerciales, pedidos, entregas, permisos, consultas y evidencias. Se representan con columnas tipadas, claves y restricciones.
Semiestructurados	Metadatos de versiones, parámetros y contenido de resultados estructurados, y detalles de auditoría. Se guardan en JSONB porque su forma puede variar dentro de límites controlados.
No estructurados	Archivos Markdown de las versiones documentales. Permanecen fuera de PostgreSQL y se vinculan mediante ruta, MIME type, tamaño y checksum.
Vectoriales	Embeddings sintéticos de los fragmentos documentales, almacenados como VECTOR(32) y asociados a una versión identificable del modelo.
Operacionales	Pedidos, líneas, condiciones comerciales, entregas, incidencias, consultas y respuestas. Sostienen la operación o registran una interacción concreta.
Analíticos	Totales por categoría, rankings por segmento, métricas de uso y planes de ejecución. En la prueba se calculan bajo demanda y no justifican un Data Warehouse separado.
Sensibles o restringidos	Condiciones comerciales, incidencias y documentos legales o de cumplimiento. Su visibilidad depende del perfil y de la clase documental.
Auditoría y trazabilidad	Eventos de consulta, acceso, publicación, recuperación y evidencia. Conservan referencias y contexto, pero no duplican el contenido sensible.

Cuadro 2: Clasificación de los datos de la solución.

se divide en fragmentos ordenados. Cada fragmento puede conservar un embedding por versión del modelo. Una respuesta exitosa se relaciona con al menos una evidencia documental o estructurada, mientras que los resultados negativos no admiten evidencia.

El historial forma parte del modelo conceptual. Las condiciones comerciales y las versiones documentales tienen vigencia temporal; una versión sustituida o revocada deja de participar en la recuperación, pero no se elimina porque puede haber sostenido una respuesta anterior.

5. Modelo de implementación según la tecnología elegida

El modelo lógico relacional y el físico se muestran en las Figuras 2 y 3. Sus fuentes canónicas son los archivos Mermaid `docs/diagramas/logico.mmd` y `docs/diagramas/fisico.mmd`. El esquema SQL es la autoridad sobre el comportamiento efectivamente implementado.

Las tablas usan claves internas **BIGINT GENERATED BY DEFAULT AS IDENTITY**; los códigos de producto, cliente, proveedor, pedido y documento son claves de negocio únicas. Las asociaciones puras usan claves primarias compuestas y las claves foráneas históricas aplican **ON DELETE RESTRICT**. Los estados cerrados se controlan mediante **CHECK**, sin crear catálogos que no aportan comportamiento.

La integridad temporal se expresa con intervalos semiabiertos y restricciones **EXCLUDE USING gist**. Esto impide que dos condiciones comerciales del mismo cliente y producto se superpongan y evita dos versiones publicadas simultáneamente para un documento. Los constraint triggers diferidos validan al cierre de la transacción que todo pedido conserve al menos una línea y que la evidencia coincida con el tipo de respuesta.

El recorrido documental es `documento` → `version_documental` → `fragmento` → `embedding`. La evidencia referencia el embedding exacto y permite reconstruir por relaciones la versión, el documento, el archivo y el modelo utilizados. Los índices B-tree acompañan joins y filtros observados; el índice HNSW cubre el acceso top-k por distancia coseno.

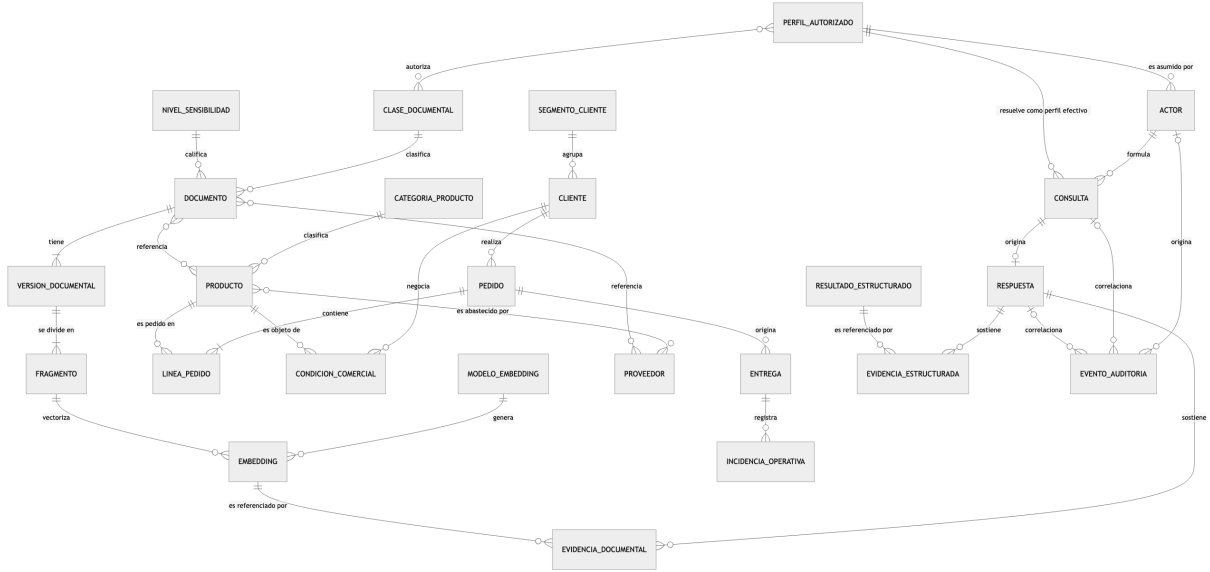


Figura 1: Modelo conceptual del dominio.

6. Decisiones de normalización, embebido, referencia y desnormalización

La información que se identifica, relaciona, autoriza o filtra con frecuencia se conserva tipada y normalizada. Categorías, perfiles, clases documentales y modelos de embedding se separan de las entidades que los usan; las relaciones N:M se materializan con tablas intermedias. Este diseño evita repetir nombres y reglas en pedidos, documentos o permisos, y permite que las claves foráneas rechacen referencias inválidas.

La desnormalización se reserva para hechos históricos. Cada línea de pedido guarda el precio y el descuento aplicados, aunque luego cambie la condición comercial. De forma análoga, un resultado estructurado conserva parámetros, contenido, instante y hash; la evidencia documental fija el embedding usado. Estos snapshots no reemplazan a las entidades vigentes: conservan el estado necesario para explicar una operación o respuesta pasada.

JSONB se usa en cuatro lugares de estructura variable acotada: `version_documental.metadata`, `resultado_estructurado.parametros`, `resultado_estructurado.contenido` y `evento_auditoria.detalles`. No se crearon índices GIN porque los nueve casos verificables no filtran por claves internas de esos objetos. Las columnas que intervienen en integridad, autorización o joins permanecen fuera de JSONB.

Los archivos Markdown no se embeben en la base. Cada versión los referencia mediante una ruta relativa portable y conserva nombre, MIME type, tamaño y SHA-256. Así, PostgreSQL controla la identidad del archivo sin convertir la tabla de versiones en almacenamiento de binarios.

7. Justificación de la tecnología seleccionada

La implementación usa PostgreSQL con `pgvector` y `btree_gist`. El dominio requiere relaciones fuertes, vigencia temporal, autorización por fila, agregaciones SQL y búsqueda vectorial. Un único motor puede aplicar esas reglas dentro de la misma transacción y evita sincronizar permisos o estados documentales con un índice externo.

`pgvector` aporta el tipo `VECTOR(32)`, la distancia coseno y el índice HNSW. `btree_gist` permite combinar igualdad e intervalos en restricciones de exclusión. PostgreSQL también aporta

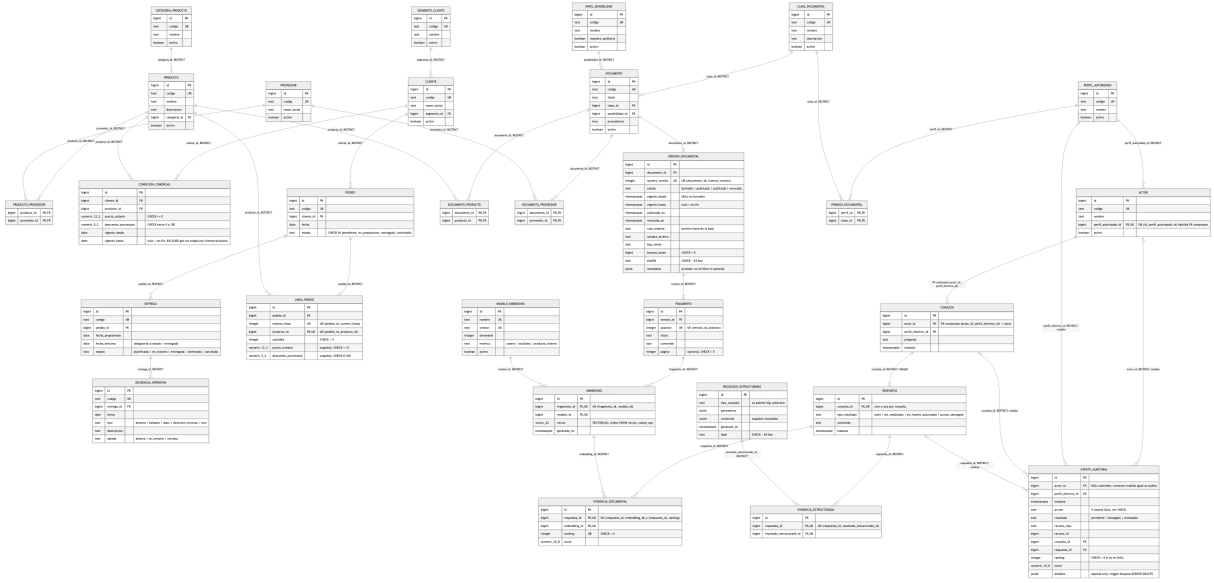


Figura 3: Modelo físico de la implementación.

segundo nivel produce un fragmento. Los oráculos fijan como primeros resultados los fragmentos 10 para la búsqueda vectorial y 18 para la recuperación híbrida de Comercial/Compras.

10. Consultas representativas

El conjunto contiene siete consultas funcionales y dos pruebas transversales. Cada caso declara parámetros, resultado esperado y orden estable mediante códigos de negocio.

C1–C7 producen los siete pares **consulta–respuesta** del conjunto. C8 y C9 reutilizan esas interacciones para comprobar aislamiento y trazabilidad; no crean dos conversaciones adicionales. Esta distinción evita confundir cantidad de casos SQL con cantidad de interacciones simuladas.

11. Propuesta para datos semiestructurados, no estructurados y vectoriales

Cada fragmento documental es la unidad vectorizada. La tabla **embedding** referencia directamente al fragmento y al modelo; la versión, el documento, la vigencia y la clase se alcanzan mediante relaciones. La vista **fragmento_recuperable** reúne esos datos sin duplicarlos en el vector. Las políticas RLS sobre las tablas base agregan el permiso del perfil efectivo.

Los vectores sintéticos tienen 32 dimensiones, están normalizados y forman vecindarios temáticos deterministas. La búsqueda usa distancia coseno mediante el operador **<=>**; ordena de menor a mayor distancia y usa **fragmento_id** como desempate. El modelo activo cubre todos los fragmentos. Cuatro fragmentos conservan además embeddings históricos, que no participan en la recuperación vigente.

El índice HNSW usa **vector_cosine_ops**. En la muestra, PostgreSQL elige un *sequential scan* porque comparar 32 vectores activos cuesta menos que recorrer el índice. Una ejecución diagnóstica con **enable_seqscan = off** confirma que el índice puede resolver el patrón top-k; no se fuerza ese plan como resultado de rendimiento.

Los riesgos principales son recuperar una versión obsoleta, exponer una clase no autorizada, mezclar modelos o presentar una respuesta sin fuente. La vista filtra actividad, publicación, vigencia y modelo; RLS limita la clase antes del **ORDER BY** y del **LIMIT**; la unicidad por fragmento

Alternativa	Ventaja posible	Motivo para no adoptarla como motor principal
Base documental	Flexibilidad para versiones y metadatos JSON.	Traslada relaciones, exclusión temporal y autorización uniforme a lógica de aplicación.
Clave-valor	Baja latencia para caché o contexto efímero.	No resuelve joins, agregaciones ni trazabilidad histórica como almacenamiento primario.
Columnar	Escritura sostenida de eventos y consultas por rango temporal.	La muestra necesita correlacionar auditoría con actores, consultas y respuestas mediante joins.
Grafo	Recorridos de relaciones variables.	Las relaciones del caso tienen profundidad conocida y se expresan con claves foráneas.
Vectorial dedicada	Mayor throughput de similitud a gran escala.	Exige replicar vigencia y permisos para filtrar antes del top-k y abre un problema de consistencia entre motores.

Cuadro 3: Alternativas consideradas frente a PostgreSQL.

Grupo	Cantidad	Componentes principales
Operación	12 productos, 12 pedidos	6 clientes, 28 líneas, 14 entregas y 4 incidencias
Documentación	10 documentos	14 versiones y 32 fragmentos
Vectores	36 embeddings	32 del modelo activo y 4 históricos
Acceso	6 actores	3 perfiles, 5 clases y 8 permisos
Interacción	7 pares	7 consultas, 7 respuestas y sus evidencias
Auditoría	56 eventos	Cambios documentales, consultas, accesos y evidencia

Cuadro 4: Escala principal del conjunto sintético.

y modelo evita duplicados; y la evidencia apunta al `embedding_id` exacto. De ese modo una respuesta histórica sigue siendo explicable aunque cambie la versión vigente.

12. Propuesta de arquitectura de datos

Para esta primera versión proponemos una arquitectura simple, con una única instancia de PostgreSQL como núcleo de la solución. En esa instancia se concentran los datos operativos, los metadatos documentales, los embeddings, los permisos, las consultas, las respuestas y la auditoría. PostgreSQL tiene las extensiones `pgvector`, para guardar y consultar embeddings, y `btree_gist`, para controlar períodos de vigencia sin solapamientos.

Elegimos un motor único porque el caso necesita combinar en una misma consulta relaciones estructuradas, vigencia de documentos, permisos y búsqueda por similitud. Si separáramos desde el inicio la base relacional y la base vectorial, tendríamos que replicar metadatos como permisos, clase documental y vigencia, y mantenerlos sincronizados. Para el alcance del TP, esa complejidad no se justifica.

El recorrido de los datos comienza con el script `scripts/generar_datos.py`, que genera datos sintéticos, documentos Markdown y un manifiesto con conteos, checksums y oráculos de consulta. Luego `scripts/cargar_base.sh` crea la base y ejecuta los scripts de esquema, datos, índices, vistas y seguridad. Los documentos se registran junto con sus versiones; cada versión conserva información de vigencia y una referencia verificable al archivo original. Después, cada versión documental se divide en fragmentos y los fragmentos se asocian a embeddings sintéticos de dimensión 32 y a un modelo de embedding.

Caso	Pregunta o control	Patrón y resultado comprobado
C1	Condición comercial vigente.	Filtra un intervalo semiabierto por cliente, producto y fecha; devuelve una fila aplicable.
C2	Pedidos y entregas que requieren atención.	Combina cuatro entidades con <code>JOIN/LEFT JOIN</code> ; incluye demoras y parciales, y excluye el pedido cancelado.
C3	Importe neto por categoría.	Usa <code>GROUP BY</code> sobre cantidad, precio y descuento; devuelve totales y conteos exactos.
C4	Productos principales por segmento.	Aplica <code>DENSE_RANK</code> ; el código de producto sólo estabiliza la presentación de empates.
C5	Clientes sin pedidos históricos.	Usa <code>NOT EXISTS</code> ; devuelve el único cliente reservado para ese escenario.
C6	Búsqueda vectorial top-k.	Ordena por distancia coseno y fragmento; el fragmento 10 ocupa la primera posición.
C7	Recuperación híbrida autorizada.	Combina similitud, vigencia, modelo activo y RLS; Comercial/Compras obtiene primero el fragmento 18 y Operaciones no ve su clase.
C8	Matriz de autorización.	Recorre las 15 combinaciones perfil-clase y confirma siete denegaciones por ausencia de permiso.
C9	Trazabilidad e inmutabilidad.	Correlaciona actor, consulta, respuesta, evidencia y auditoría; las modificaciones operacionales de eventos son rechazadas.

Cuadro 5: Siete consultas y dos pruebas transversales.

Una consulta de recuperación usa la vista de fragmentos recuperables, que considera la actividad del documento y del modelo, el estado publicado y la vigencia. Las políticas RLS agregan el permiso del perfil efectivo antes del ranking vectorial. Una respuesta exitosa registra la evidencia documental o estructurada utilizada. Los eventos correlacionados identifican la consulta, la respuesta, los fragmentos recuperados y la evidencia; el diseño no interpreta su orden de inserción como un flujo de aplicación.

Los archivos Markdown se mantienen fuera de la base de datos. La base guarda solamente la ruta relativa y los datos necesarios para verificar su integridad. Esta decisión simplifica el modelo físico de la prueba y evita guardar binarios en tablas. En una evolución del sistema, los archivos podrían trasladarse a almacenamiento de objetos, manteniendo el mismo contrato de checksum y procedencia.

No incorporamos un Data Warehouse, Data Lake o Lakehouse porque las consultas del TP son principalmente operativas y transaccionales. Tampoco se requiere un proceso analítico masivo ni se cuenta con fuentes heterogéneas de gran volumen. Si en el futuro se necesitaran indicadores históricos complejos, análisis de uso del copiloto o reportes de negocio a gran escala, sería razonable extraer los eventos y hechos necesarios hacia una capa analítica separada. Esa separación no es necesaria para demostrar el diseño actual.

13. Estrategia de seguridad, permisos y aislamiento

Los perfiles funcionales son Operaciones y Logística, Comercial y Compras, y Administración y Calidad. Cada actor pertenece a uno solo. El permiso se modela entre perfil y clase documental; una combinación ausente queda denegada. El nivel de sensibilidad describe el dato, pero no reemplaza esa matriz.

Los roles técnicos separan responsabilidades: `rag_ingesta` incorpora borradores y vectores; `rag_documental` publica o revoca mediante funciones atómicas; `rag_runtime` consulta y registra interacciones; y `rag_auditor` reconstruye evidencia en modo de lectura. Son roles `NOLOGIN`, sin privilegios de superusuario ni `BYPASSRLS`. La autenticación y la propagación de credenciales pertenecen a una aplicación futura y quedan fuera del TP.

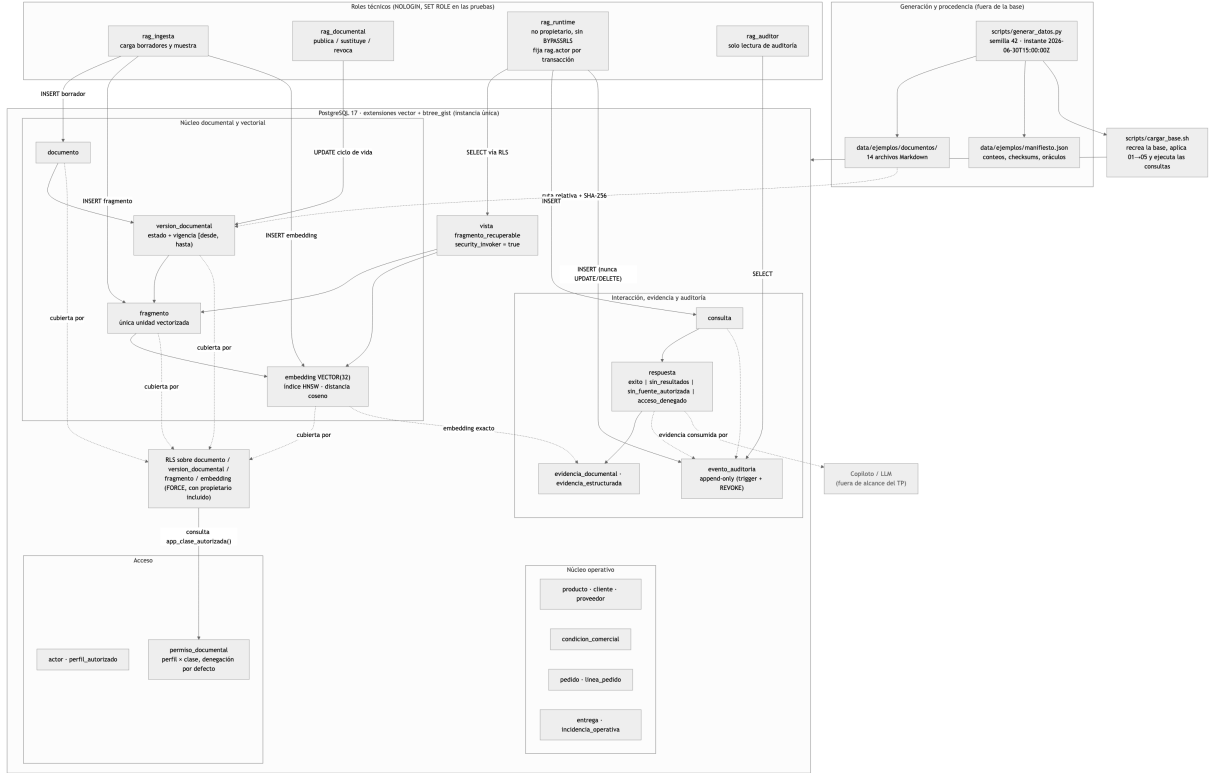


Figura 4: Arquitectura general y flujo de datos.

Cada transacción fija el código del actor en `rag.actor`. Las funciones de contexto verifican que exista, esté activo y tenga un perfil activo. Un valor ausente, vacío, inexistente o inactivo devuelve un contexto nulo y las políticas deniegan la lectura. La base deriva el perfil desde el actor y evita que ambos valores se contradigan; no autentica, en cambio, quién fija `rag.actor`. Esa responsabilidad queda en la futura capa de aplicación, que deberá asociar la identidad autenticada con el actor de cada transacción.

RLS se aplica y fuerza sobre `documento`, `version_documental`, `fragmento` y `embedding`. También limita condiciones comerciales e incidencias según el perfil. La vista de recuperación usa semántica de invocador, por lo que no ejecuta con privilegios del propietario. Las 15 combinaciones perfil-clase, los cuatro contextos inválidos y el acceso directo a tablas se comprueban desde un rol no propietario.

La auditoría es append-only para los roles operacionales: no pueden insertar eventos arbitrarios, modificarlos ni eliminarlos. Los triggers generan eventos para consultas, respuestas y evidencias; las funciones documentales registran publicación, sustitución y revocación dentro de la misma transacción. El propietario conserva `TRUNCATE` para recrear el conjunto sintético, una operación administrativa que no se concede a los roles de aplicación.

14. Consideraciones de escalabilidad y rendimiento

La implementación usa una muestra pequeña y su objetivo es validar el diseño, no medir rendimiento de producción. La tabla más grande de la carga tiene 56 filas y la tabla de embeddings tiene 36 filas. Por este motivo, los tiempos observados y los planes de ejecución no se deben extrapolar directamente a un sistema real con muchos usuarios y documentos.

Las estructuras que crecerían más en un escenario real son `evento_auditoria`, `consulta`, `respuesta`, `fragmento` y `embedding`. La auditoría crece con cada acción relevante y no se elimina, por lo que es la candidata más clara para aplicar particionado por rango de fecha. Por

ejemplo, se podrían crear particiones mensuales o trimestrales de `evento_auditoria`, según el volumen de escritura y el período de retención requerido.

Los fragmentos y embeddings crecerían a medida que se incorporen documentos o nuevas versiones. En el modelo actual, cada fragmento puede tener un embedding por modelo. Esto permite conservar vectores históricos, pero también implica que una migración de modelo de embedding aumenta temporalmente el volumen de la tabla. Una política de retención de modelos antiguos y la separación entre modelos activos e históricos ayudarían a controlar ese crecimiento.

Las consultas más críticas son las búsquedas vectoriales top-k y las búsquedas híbridas autorizadas. Estas últimas combinan similitud por coseno, estado del documento, vigencia y permisos. Para la búsqueda por similitud se creó un índice HNSW sobre el vector, usando `vector_cosine_ops`. El índice está justificado por el patrón de recuperación esperado cuando el corpus crece, aunque en la muestra el optimizador elija un *sequential scan*: recorrer 36 vectores es más barato que entrar al índice. La evidencia de este comportamiento está documentada en `evidencias/planes_ejecucion.md`.

También se definieron índices B-tree para los filtros y relaciones usadas por las consultas representativas: condiciones comerciales por cliente y producto, entregas e incidencias, relaciones de pedidos, filtros de vigencia documental y correlaciones de auditoría. No se agregaron índices para todas las columnas, porque cada índice mejora algunas lecturas pero agrega espacio y costo en las escrituras.

Una primera estrategia de escalabilidad sería aumentar los recursos de la instancia de PostgreSQL y revisar periódicamente los planes de ejecución. Si el tráfico de lectura creciera, se podrían agregar réplicas de sólo lectura para reportes o consultas que no requieran consistencia inmediata. Las operaciones de publicación, permisos y auditoría deberían mantenerse en la instancia principal para conservar la consistencia transaccional.

En una etapa posterior se podrían separar algunos componentes. Los archivos originales podrían ir a almacenamiento de objetos; los eventos de auditoría podrían particionarse o moverse a una solución orientada a escritura masiva; y una base vectorial dedicada podría evaluarse si la cantidad de embeddings y el throughput de búsquedas crecieran varios órdenes de magnitud. Sin embargo, esa última alternativa requeriría mantener sincronizados los permisos y la vigencia con el índice vectorial, por lo que agrega complejidad y riesgo de inconsistencia.

La separación de componentes debe responder a mediciones: crecimiento por tabla, latencia del top-k, costo de escritura de auditoría y tolerancia a consistencia diferida. Hasta que esos datos muestren un límite, particionar o replicar dentro de PostgreSQL conserva las garantías actuales con menos coordinación entre sistemas.

15. Conclusiones

El trabajo materializa una capa de datos pequeña para un copiloto RAG interno. El mismo núcleo relacional conserva hechos operativos, documentos versionados, embeddings, permisos, interacción y auditoría. Esta decisión permite que vigencia y autorización reduzcan el universo antes del ranking vectorial y que cada respuesta conserve la evidencia exacta que la sostuvo.

La prueba funcional demuestra claves y restricciones, invariantes diferidas, publicación documental atómica, RLS, privilegios mínimos, auditoría operacional append-only y nueve casos verificables. El generador produce los mismos archivos, conteos y checksums con semilla 42; la validación recrea dos veces la base y compara sus snapshots. Los planes muestran que un *sequential scan* es correcto para 36 embeddings, aunque el índice HNSW esté disponible para el patrón de crecimiento previsto.

Los límites son deliberados: no hay autenticación, API, LLM, embeddings reales ni infraestructura distribuida. Tampoco se usa la muestra como benchmark. Una evolución debería medir primero el crecimiento y, según el cuello observado, particionar auditoría, agregar réplicas de lectura, mover archivos a almacenamiento de objetos o evaluar un índice vectorial separado.

Cualquiera de esas decisiones deberá preservar la autorización previa al ranking y la trazabilidad histórica.

A. Reproducibilidad y material de apoyo

La carga y validación se reproducen con las instrucciones del `README.md`. Las salidas extensas permanecen en el repositorio para que el informe pueda interpretarlas sin copiarlas.

Artefacto	Qué permite comprobar
Manifiesto JSON	Procedencia, semilla, instante de referencia, conteos, checksums y oráculos vectoriales.
Validaciones SQL	Rechazo de operaciones inválidas, aislamiento, ciclo documental y auditoría.
Matriz de autorización	Quince combinaciones perfil-clase, contextos inválidos, autorización previa al ranking y trazabilidad.
Planes de ejecución	Planes observados con HNSW disponible, uso diagnóstico del índice e interpretación de los B-tree.
Matriz de cobertura	Correspondencia entre consigna, informe, implementación y evidencia.

Cuadro 6: Evidencias reproducibles del repositorio.

Las rutas correspondientes son `data/ejemplos/manifiesto.json`, `db/consultas/07_vali`
`daciones.sql`, `evidencias/seguridad/matriz_autorizacion.md`, `evidencias/planes_ej`
`ecucion.md` y `docs/matriz_cobertura.md`.